

1.2. Метрики Джилба

1.2.1. Теоретические сведения

Достаточно практичными являются метрики программного обеспечения, предложенные Томасом Джилбом (Thomas Jilb) и также основанные на результатах анализа текстов программных продуктов.

В качестве меры логической трудности Джилб предложил число логических «двоичных принятий решений». Наиболее ценным для практики является то, что такая оценка может быть получена вручную на основе зрительного анализа текста программы либо автоматически с помощью специально разработанных программных анализаторов, причем относительно несложных.

Абсолютная логическая сложность, по мнению автора метрик, должна задаваться числом необычных выходов из операторов, в которых происходит принятие решений. Джилб предположил, что логическая сложность должна являться значимым, если не определяющим, фактором для оценки стоимости программы на начальных этапах ее проектирования.

Логическая сложность программы Джилб определяет как насыщенность программы условными операторами типа IF–THEN–ELSE и операторами цикла (при этом следует учитывать, что фактическая запись условий и циклов в разных языках программирования может быть представлена в разной форме при сохранении указанного смысла операторов). При этом вводятся следующие характеристики программного средства:

- CL – абсолютная сложность программы, характеризуемая количеством операторов условий;
- cl – относительная сложность программы, определяющая насыщенность программы операторами условия (т. е. относительная сложность программы cl вычисляется как отношение абсолютной сложности CL к общему числу операторов L).

Кроме указанных показателей Джилб предложил применять следующие характеристики программы:

- количество операторов цикла L_{loop} ;
- количество операторов условия L_{IF} ;
- число модулей или подсистем L_{mod} ;

- отношение числа связей между модулями к числу модулей

$$f = \frac{N_{sv}^4}{L_{mod}}; \quad (1.2.1)$$

- отношение числа ненормальных выходов из множества операторов к общему числу операторов

$$f^* = \frac{N_{sv}^*}{L}. \quad (1.2.2)$$

Среди всех показателей качества программ Джилб указывает надежность программы, которую он характеризует как возможность того, что данная программа проработает определенный период времени без логических сбоев. В качестве практической оценки программной надежности автор метрик предлагает рассчитывать как единицу минус отношение числа логических сбоев к общему числу запусков.

Отношение количества правильных данных ко всей совокупности данных приводится Джилбом в качестве меры точности (свободы от ошибок), поскольку он считает, что точность нужна как средство обеспечения надежности программы. Прецизионность определяется как мера того, насколько часто появляются ошибки, вызванные одинаковыми причинами. Джилб оценивает этот показатель дробью, в числителе которой указывается число фактических ошибок на входе, а в знаменателе – общее число наблюдаемых ошибок, причинами которых явились эти ошибки на входе. Так, например, если одна ошибка вызывает в течение определенного периода времени появление 50 сообщений об ошибках, то прецизионность равна 0,02.

1.2.2. Задача «Вычисление значений функции»

Функция $Y=F(x)$ задана следующим образом:

$$Y = \begin{cases} 0,5 & \text{при } x \leq 0,5; \\ x+1 & \text{при } -0,5 < x \leq 0; \\ x^2 - 1 & \text{при } 0 < x \leq 1; \\ x-1 & \text{при } x > 1. \end{cases}$$

Необходимо разработать программу для вычисления значений этой функции и на основе лексического анализа исходного текста программы оценить ее качество с использованием метрик Джилба.

Реализация программы

Текст программы для реализации возможного алгоритма решения поставленной задачи, разработанный с использованием языка программирования C#, приведен на рис. 1.7.

Номера строк	Строки программы
1	using System;
2	
3	class Operator
4	{
5	public static void Main()
6	{
7	float x,
8	y;
9	char rep;
10	string str;
11	
12	REPEAT:
13	Console.Clear();
14	Console.Write("Введите аргумент функции: ");
15	str = Console.ReadLine();
16	x = float.Parse(str);
17	if (x <= 0)
18	if (x <= -0.5)
19	y = (float)0.5;
20	else
21	y = (float)(x + 1.0);
22	else
23	if (x <= 1.0)
24	y = (float)(x * x - 1.0);
25	else
26	y = (float)(x - 1.0);
27	
28	str = "F(" + x + ")=" + y;
29	Console.WriteLine(str);
30	
31	Console.Write("Для повтора вычислений нажмите
32	клавишу Y: ");

33	rep = char.Parse(Console.ReadLine());
34	if (rep == 'Y' rep == 'y') goto REPEAT;
35	}
36	}

Рис. 1.7. Текст программы для расчета значений функций $Y=F(x)$

Программа написана без использования операторов цикла. Повторение процедур выполнения операторов программы реализовано с помощью оператора *goto*, применение которого не рекомендуется при разработке профессионального программного обеспечения, поскольку затрудняет понимание программы. Однако в учебных целях применение данного оператора будем считать допустимым, тем более что такой оператор не влияет на параметры оценки ее качества с использованием метрик Джилба.

Программа, разработанная для реализации заданного алгоритма, не имеет циклов и модулей. Следовательно, из перечисленного набора характеристик можно определить лишь следующие: L , L_{IF} , CL и cl , причем для такого случая $CL = L_{IF}$.

Словарь программы

При подсчете общего количества операторов L программы будем руководствоваться правилами, определенными при подсчете операторов в метриках Холстеда.

В табл. 1.22 приведены операторы и операции, используемые в программе. Структура таблицы аналогична рассмотренным ранее в разд. 1.1. При подсчете числа операторов следует иметь в виду, что в строке 28 (см. рис. 1.7) скобки используются как символы строковых констант, а потому операторами не являются.

Таблица 1.22. Словарь операторов и операций программы

№ п/п	Операторы, операций	Номера строк	Количество повторений
1	using ...	1	1
2	class ...	3	1
3	{}	4(35), 6(34)	2
4	public static void	5	1
5	float	7	1
6	char	9	1

Окончание табл. 1.22

№ п/п	Операторы, операции	Номера строк	Количество повторений
7	string	10	1
8	Console.Clear()	13	1
9	Console.Write()	14, 31	2
10	=	15, 16, 19, 21, 24, 26, 28, 32	8
11	Console.ReadLine()	15, 32	2
12	Parse	16, 32	2
13	if () ... else...	17(22), 18(20), 23(25)	3
14	if ()...	33	1
15	<=	17, 18, 23	3
16	+	21, 28	2
17	-	24, 26	2
18	*	24	1
19	Console.WriteLine()	29	1
20	==	33	2
21		33	1
22	goto	33	1
23	;	1, 8, 9, 10, 13, 14, 15, 16, 19, 21, 24, 26, 28, 29, 31, 32, 33	17
24	,	7	1
25	.	13, 14, 15, 16, 18, 19, 21, 23, 24, 26, 29, 31, 32, 32,	14
26	()	5, 13, 14, 15, 16, 17, 18, 19, 21, 21, 23, 24, 24, 26, 26, 29, 31, 32, 32, 33	20
Всего			92

Оценка характеристик программы

Согласно теории Джилба необходимо определить количество операторов условия, используемых в исходном тексте программы для реализации алгоритма решения поставленной задачи.

В языке программирования С# к такой категории инструкций могут относиться условные операторы ветвления *if* и операторы циклов *for ... while* и *do ... while*, которые в своем составе в обязательном порядке содержат логическое выражение, являющееся условием выполнения указанных операторов. В зависимости от соблюдения условия программа может существенно изменить ход выполнения, поэтому значимость условных операторов достаточно велика. Именно поэтому автор для них определил отдельную метрику.

В исходном тексте разработанной программы используются условные операторы *if*, которые располагаются в строках с номерами 17, 18, 23 и 33 (см. рис.1.7). Исходя из полученных данных (табл. 1.22) получаем следующие результаты оценки характеристик программы на основе метрик Джилба:

- для представленного варианта решения число операторов условий L_{IF} равно 4 (табл. 1.22, п. 13 и 14);
- абсолютная сложность $CL = 4$, так как в программе используется четыре оператора условия;
- относительная сложность программы равна:

$$cl = CL / L = 4 / 92 = 0,0435.$$

С точки зрения лексического анализа исходного текста программы представленное решение не является сложным, так как количество ветвлений в программе весьма невелико (4 оператора условия), что подтверждается невысокой величиной метрики относительной сложности программы.

1.2.3. Задача «Функция копирования элементов массива»

Необходимо разработать функцию, которая копирует положительные элементы из одного одномерного целочисленного массива в другой массив. Используя этот метод, следует выполнить копирование положительных элементов двух исходных массивов *A* и *B* в массив *C*.

Размер исходных массивов *A* и *B* ввести с клавиатуры. Эти массивы заполнить случайными числами из диапазона от -100 до 300.

Сформированный массив *C* вывести на экран. Выдать сообщение на экран в случае, когда массив *C* оказывается пустым. Для разработанной программы на основе лексического анализа исходного текста определить значения метрик Джилба.

Реализация программы

Рассмотрим вариант реализации возможного алгоритма решения поставленной задачи, разработанный с использованием языка программирования *C#*, в котором применяются программные модули.

Пример реализации такой программы приведен на рис. 1.8.